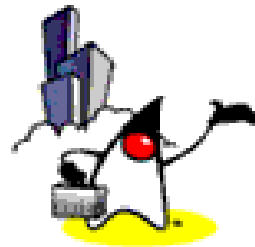




Servlet Request and Response



Request & Response



The diagram consists of two vertically stacked rounded rectangular boxes. The top box is light blue with the text 'Servlet Request' in white. The bottom box is a darker blue with the text 'Servlet Response' in white. Both boxes are enclosed within a larger, dashed-line border that follows their shape.

Servlet Request

Servlet Response

Request & Response

Servlet Request

Servlet Response

An object of `ServletRequest` is used to provide the client request information to a servlet such as content type, content length, parameter names and values, header informations, attributes etc.

Request & Response

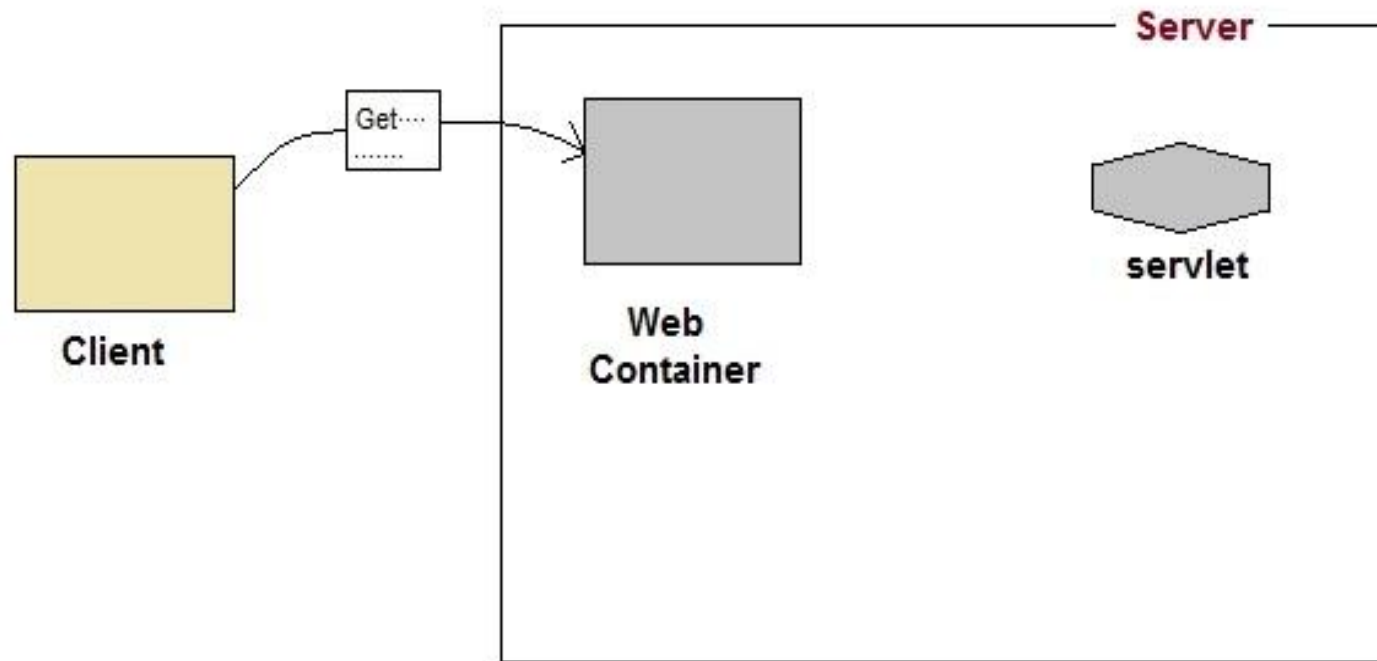
Servlet Request

Servlet Response

An object of `ServletResponse` is used to write the information back to the client.

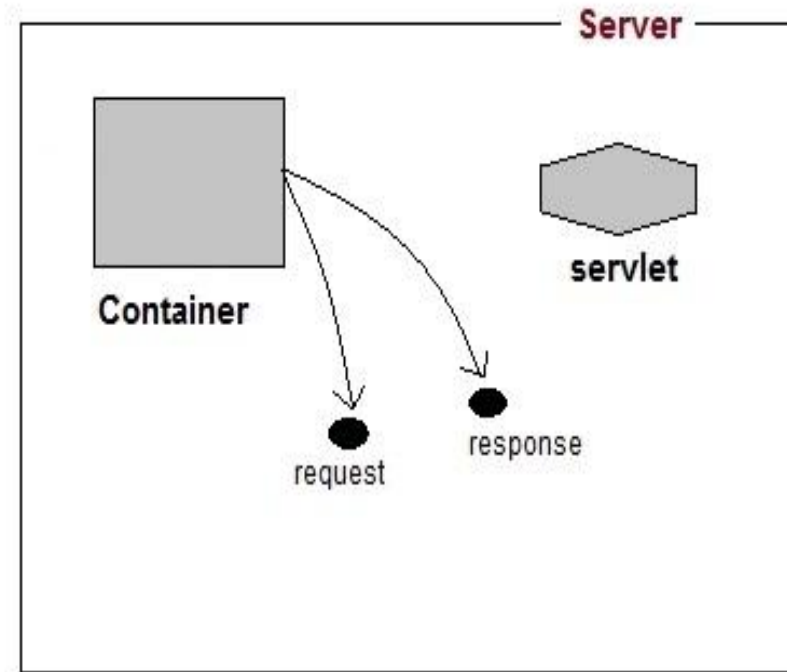
Servlet Request & Response – Scope Objects

- User sends request for a servlet by clicking a link that has URL to a servlet.



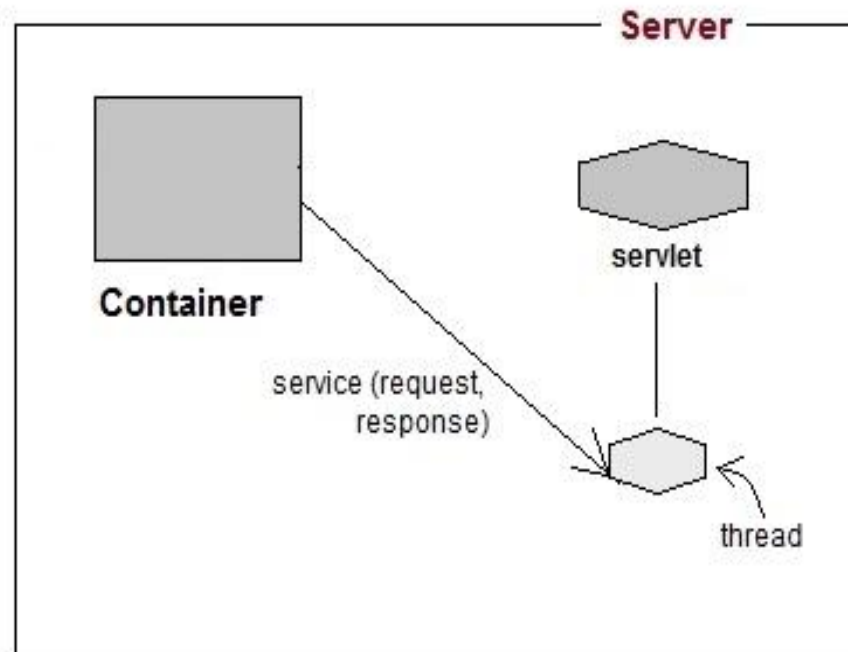
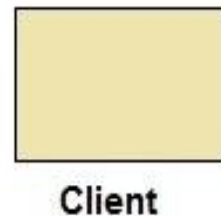
Servlet Request & Response

- The container finds the servlet using deployment descriptor/`@WebServlet` Annotation and creates two objects :
 - `HttpServletRequest`
 - `HttpServletResponse`



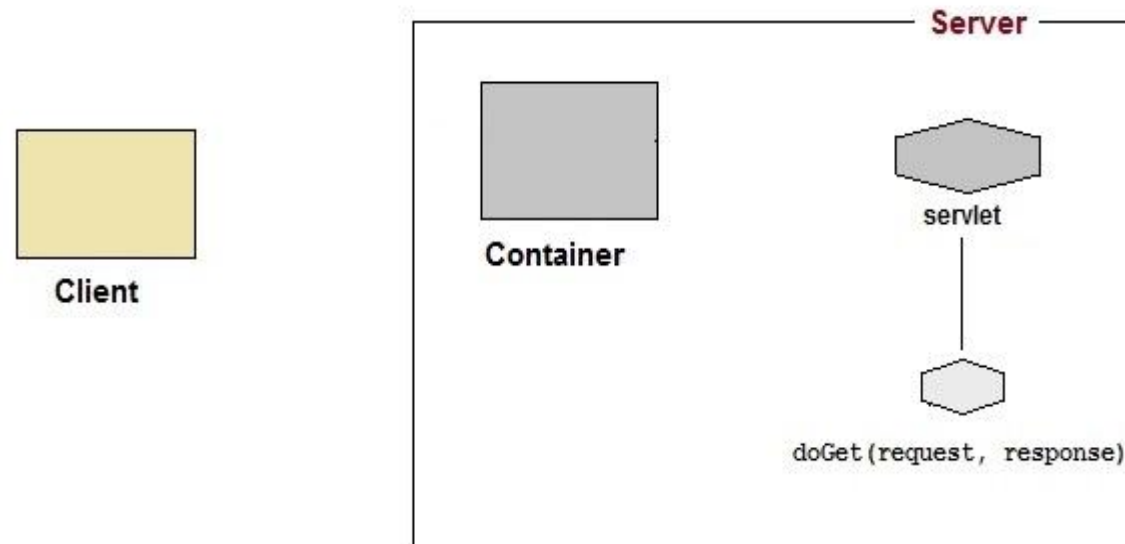
Servlet Request & Response

- Then the container creates or allocates a thread for that request and calls the Servlet's service() method and passes the request, response objects as arguments.



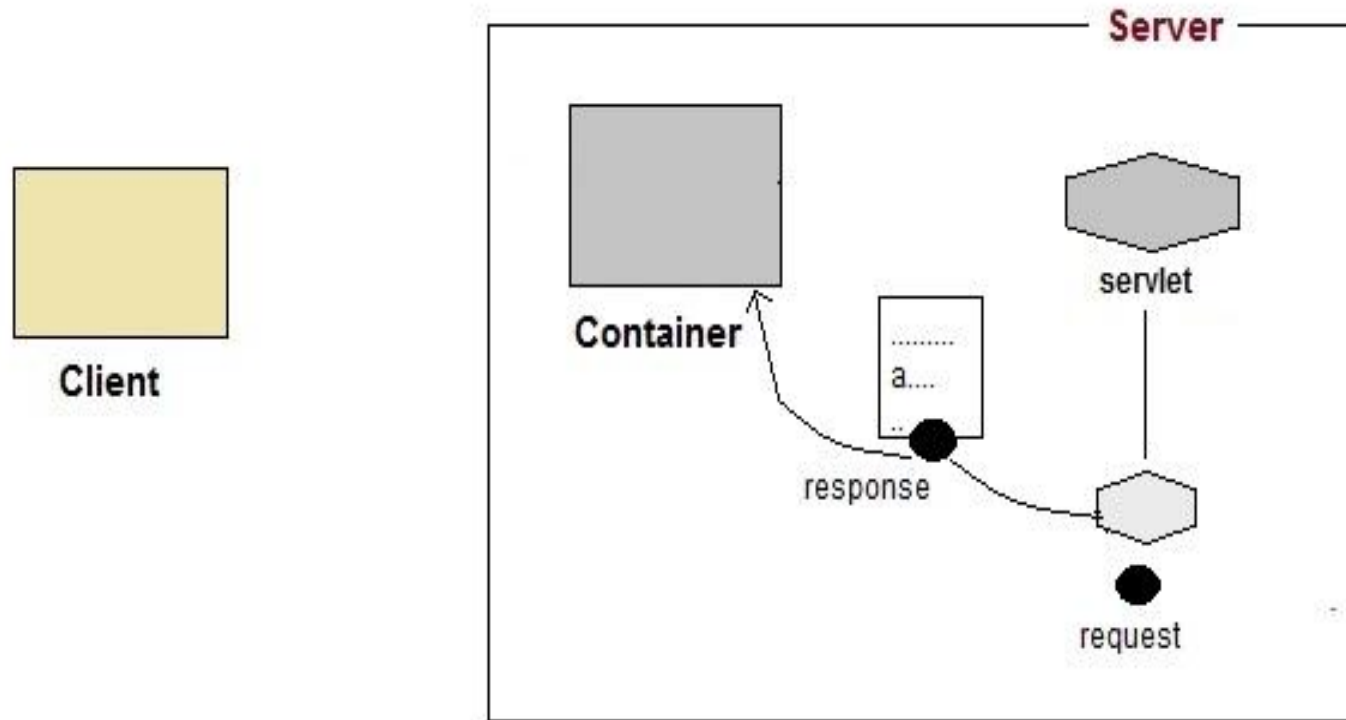
Servlet Request & Response

- The service() method, then decides which servlet method, doGet() or doPost() to call, based on HTTP Request Method(Get, Post etc) sent by the client.
- Suppose the client sent an HTTP GET request, so the service() will call Servlet's doGet() method



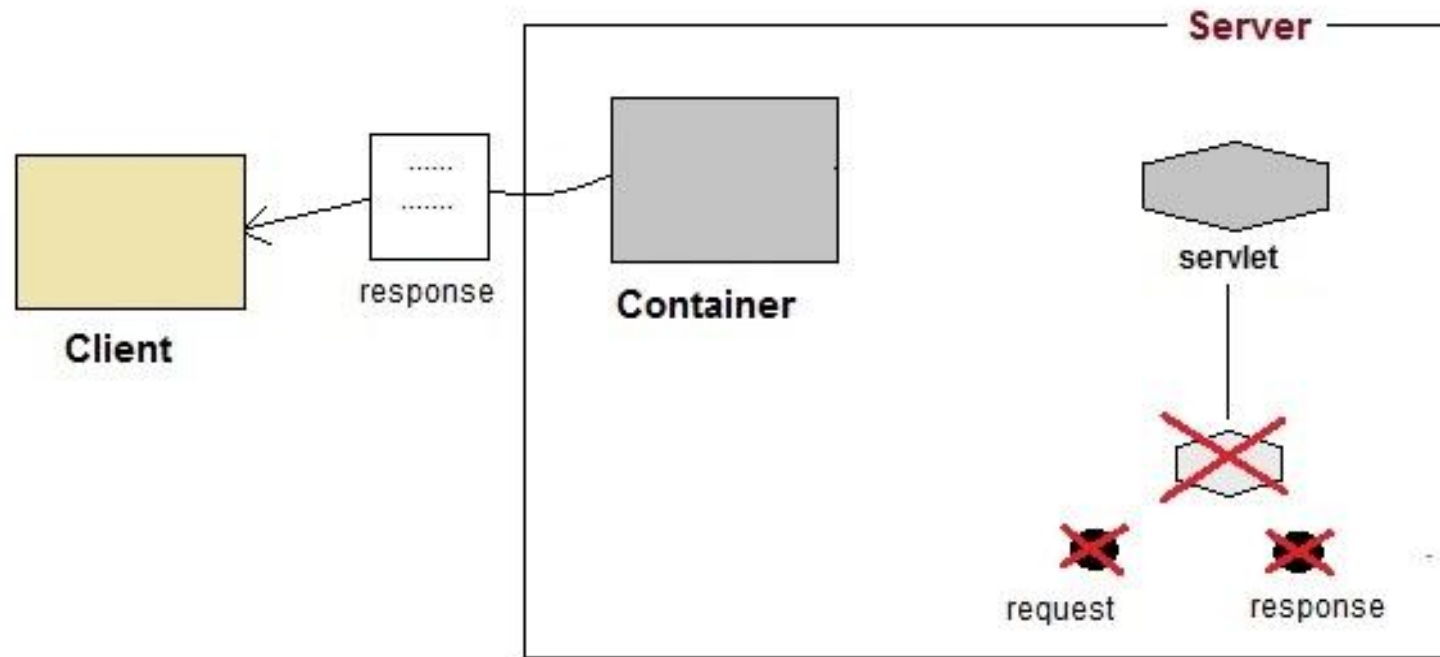
Servlet Request & Response

- Then the Servlet uses response object to write the response back to the client.



Servlet Request & Response

- After the service() method is completed the thread dies. And the request and response objects are ready for garbage collection.

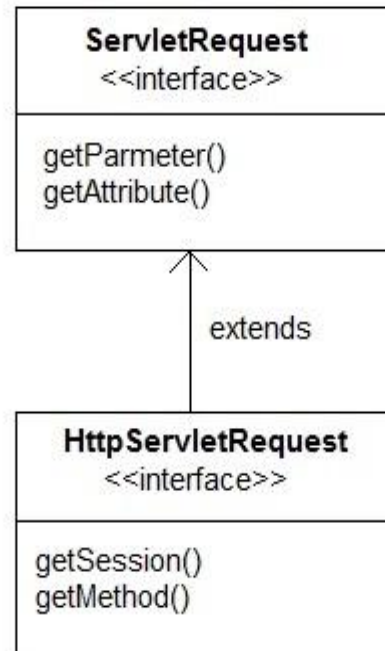


Introduction to Servlet Request

- Servlet is used to handle client request.
- Servlet API provides two important interfaces `javax.servlet.ServletException` and `javax.servlet.http.HttpServletRequest` to encapsulate client request.
- Implementation of these interfaces provide important information about client request to a servlet.
- The various methods are :
 - `getParameterValues(String name)` ,
 - `getParameter(String name);`
 - `getServletContext(),`
 - `getServerName(),`
 - `setAttribute(String name, Object o),`
 - `getAttribute(String name)`

HttpServletRequest interface

- HttpServletRequest interface adds the methods that relates to the HTTP protocol.



Important methods of HttpServletRequest

Methods	Description
String getContextPath()	returns the portion of the request URI that indicates the context of the request
Cookies getCookies()	returns an array containing all of the Cookie objects the client sent with this request
String getQueryString()	returns the query string that is contained in the request URL after the path
HttpSession getSession()	returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session
String getMethod()	Returns the name of the HTTP method with which this request was made, for example, GET, POST, or PUT.
Part getPart(String name)	gets the Part with the given name
String getPathInfo()	returns any extra path information associated with the URL the client sent when it made this request.
String getServletPath()	returns the part of this request's URL that calls the servlet

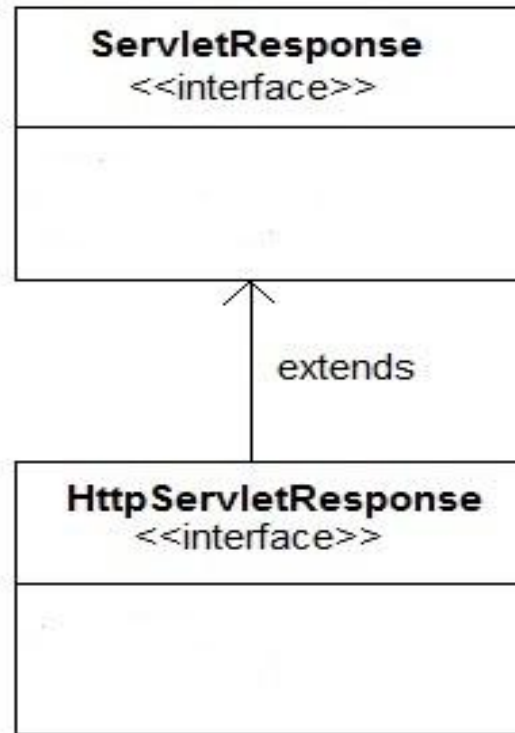
Introduction to Servlet Response

- Servlet API provides two important interfaces `ServletResponse` and `HttpServletResponse` to assist in sending response to client.

Methods	Description
<code>PrintWriter getWriter()</code>	returns a <code>PrintWriter</code> object that can send character text to the client.
<code>void setBufferSize(int size)</code>	Sets the preferred buffer size for the body of the response
<code>void setContentLength(int len)</code>	Sets the length of the content body in the response In HTTP servlets, this method sets the HTTP Content-Length header
<code>void setContentType(String type)</code>	sets the content type of the response being sent to the client before sending the respond.
<code>void setBufferSize(int size)</code>	sets the preferred buffer size for the body of the response.
<code>boolean isCommitted()</code>	returns a boolean indicating if the response has been committed
<code>void setLocale(Locale loc)</code>	sets the locale of the response, if the response has not been committed yet.

HttpServletResponse Interface

- HttpServletResponse interface adds the methods that relates to the HTTP response.



Methods of HttpServletResponse

Methods	Description
<code>void addCookie(Cookie cookie)</code>	adds the specified cookie to the response.
<code>void sendRedirect(String location)</code>	Sends a temporary redirect response to the client using the specified redirect location URL and clears the buffer
<code>int getStatus()</code>	gets the current status code of this response
<code>String getHeader(String name)</code>	gets the value of the response header with the given name.
<code>void setHeader(String name, String value)</code>	sets a response header with the given name and value
<code>void setStatus(int sc)</code>	sets the status code for this response
<code>void sendError(int sc, String msg)</code>	sends an error response to the client using the specified status and clears the buffer

Servlet Communication Methods

- The **RequestDispatcher** interface provides the facility of dispatching the request to another resource it may be html, servlet or jsp.
- This interface can also be used to include the content of another resource also.
- It is one of the way of servlet collaboration.

Servlet Communication Methods

- ❑ Servlets access network resources to satisfy client requests
- ❑ The `forward()` and `include()` methods of `RequestDispatcher` interface are used by Servlets to access network resources.
- ❑ `RequestDispatcher` interface is part of `javax.servlet` package

```
public void forward (ServletRequest req, ServletResponse res);
```

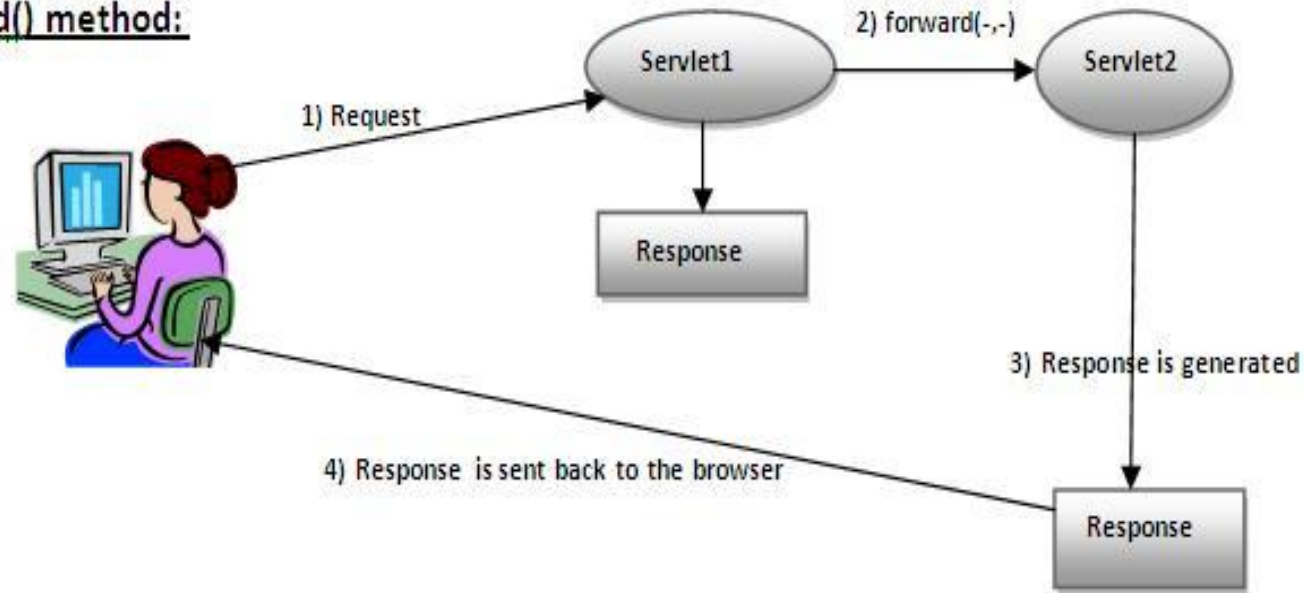
Forwards the requests from one Servlet to another on the same server

```
public void include (ServletRequest req, ServletResponse res);
```

Includes the contents of one Servlet in another Servlet in the Response

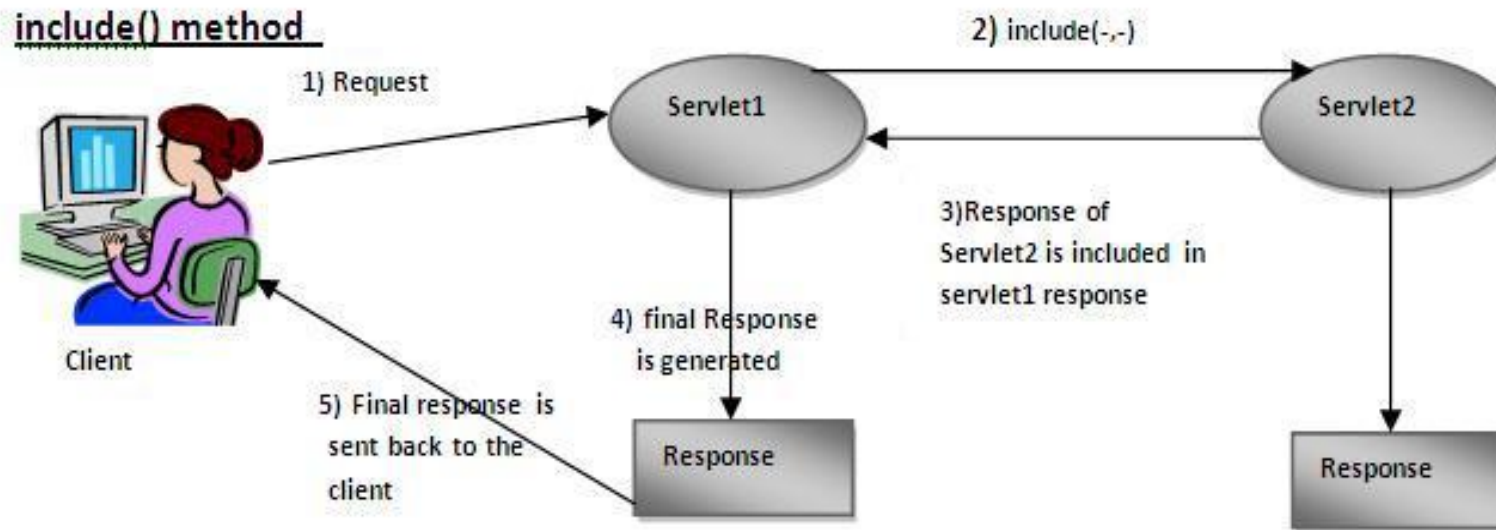
Forward() Method

forward() method:



- As you see in the above figure, response of second servlet is sent to the client.
- Response of the first servlet is not displayed to the user

Include() Method



- As you can see in the above figure, response of second servlet is included in the response of the first servlet that is being sent to the client.

Get the object of RequestDispatcher

- The **getRequestDispatcher()** method of ServletRequest interface returns the object of RequestDispatcher.
- **Syntax of getRequestDispatcher method**
public RequestDispatcher getRequestDispatcher(String resource);
- **Example of using getRequestDispatcher method**

```
RequestDispatcher rd=request.getRequestDispatcher("servlet2");  
//servlet2 is the url-pattern of the second servlet  
rd.forward(request, response); //method may be include or forward
```

SendRedirect ()-Servlet Communication

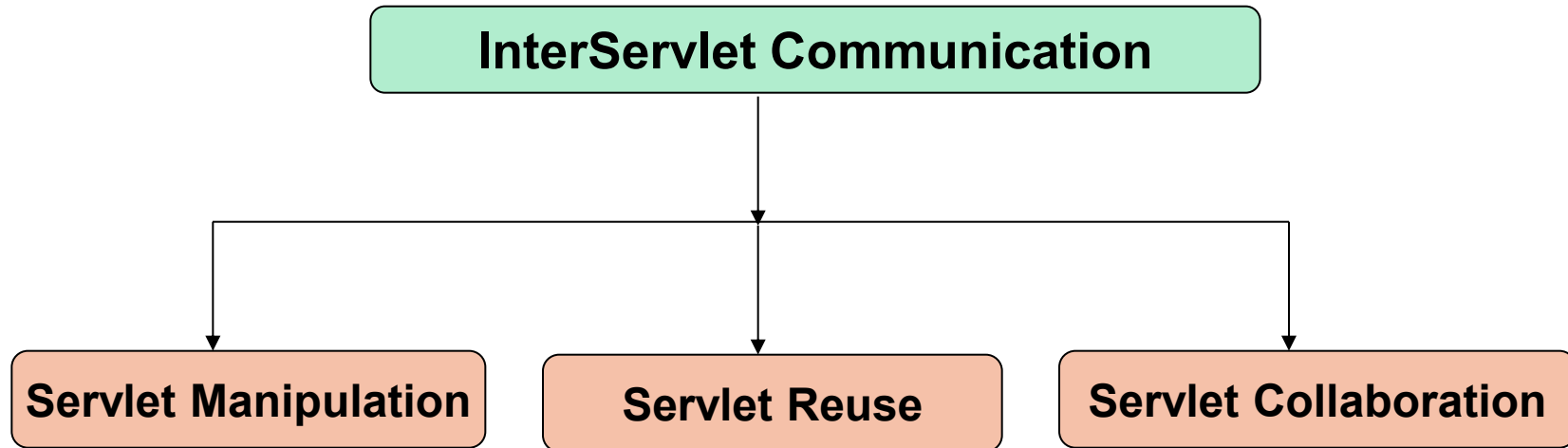
- The **sendRedirect()** method of **HttpServletResponse** interface can be used to redirect response to another resource, it may be servlet, jsp or html file.
- It accepts relative as well as absolute URL.
- It works at client side because it uses the url bar of the browser to make another request.
- So, it can work inside and outside the server.

Difference between forward() and sendRedirect() method

forward() method	sendRedirect() method
The forward() method works at server side.	The sendRedirect() method works at client side.
It sends the same request and response objects to another servlet.	It always sends a new request.
It can work within the server only.	It can be used within and outside the server.
Example: <code>request.getRequestDispatcher("servlet2").forward(request,response);</code>	Example: <code>response.sendRedirect("servlet2");</code>

InterServlet Communication 3-1

❑ InterServlet communication can be used by Servlets to gain access to other currently loaded Servlets and perform some task on each Servlet. This is called **Servlet Chaining**. The output of one Servlet act as Input to another Servlet. The InterServlet communication methods are:



Servlet collaboration allows the Servlets to share information.

InterServlet Communication

- Servlets running together in the same server have several ways to communicate with each other.

There are three major reasons to use interservlet communication:

Direct servlet manipulation:

- A servlet can gain access to the other currently loaded servlets and perform some task on each.
- The servlet could, for example, periodically ask every servlet to write its state to disk to protect against server crashes.

InterServlet Communication

Servlet reuse:

- One servlet can use another's abilities to perform a task.
- For Eg Chat Servlet written as a server for chat applets, can be reused (unchanged) by another servlet that needed to support an HTML-based chat interface.

Servlet collaboration

- The most common, situation involves two or more servlets sharing state information.
- For example, a set of servlets managing an online store could share the store's product inventory count. Session tracking is a special case of servlet collaboration.

ServletContext Interface

- An object of ServletContext is created by the web container at time of deploying the project.
- This object can be used to get configuration information from web.xml file.
- There is only one ServletContext object per web application.
- If any information is shared to many servlet, it is better to provide it from the web.xml file using the **<context-param>** element.

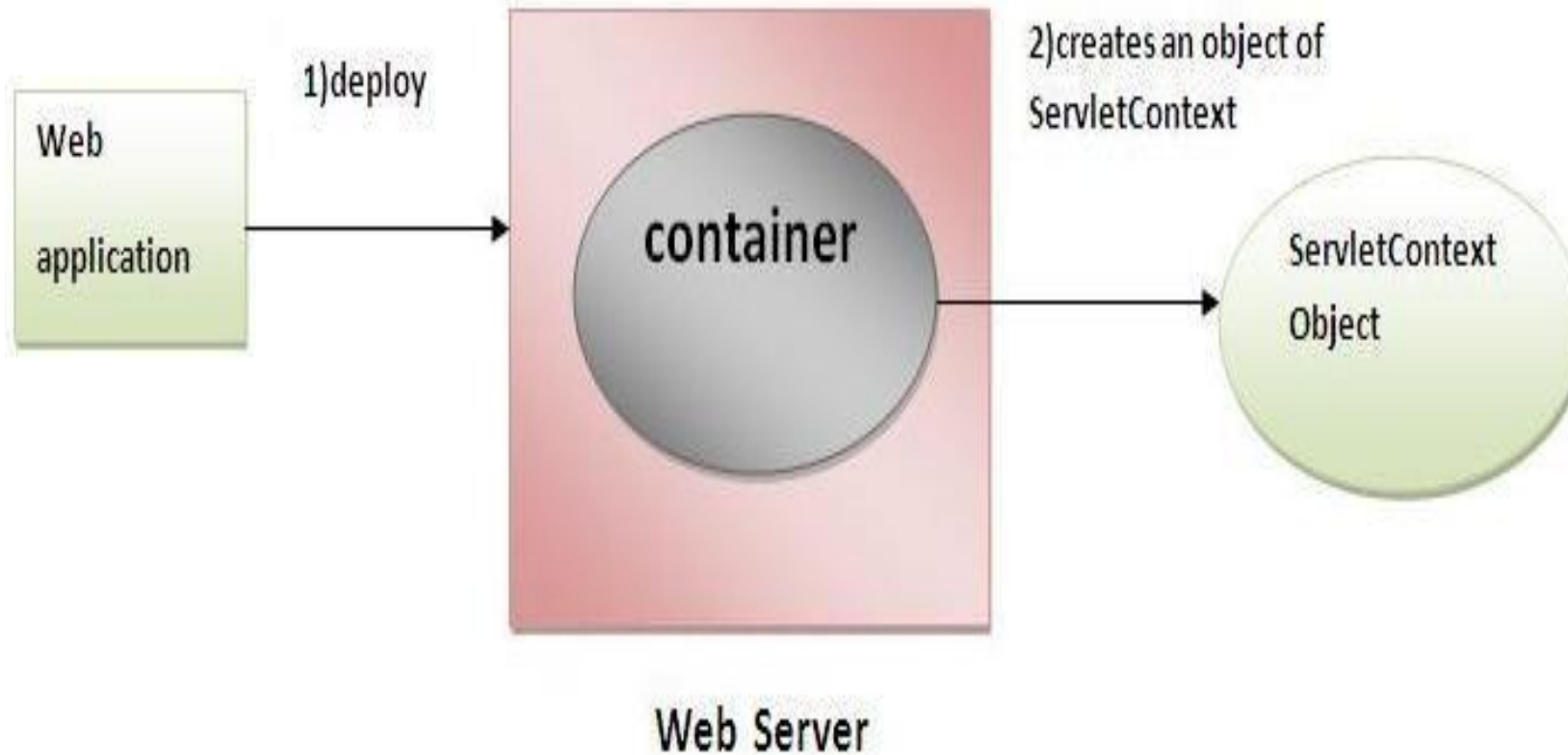
Advantage of ServletContext

- **Easy to maintain** if any information is shared to all the servlet, it is better to make it available for all the servlet.
- We provide this information from the web.xml file, so if the information is changed, we don't need to modify the servlet.
- Thus it removes maintenance problem.

Uses of ServletContext Interface

- The object of ServletContext provides an interface between the container and servlet.
- The ServletContext object can be used to get configuration information from the web.xml file.
- The ServletContext object can be used to set, get or remove attribute from the web.xml file.
- The ServletContext object can be used to provide inter-application communication.

ServletContext object



Methods of ServletContext interface

- **public String getInitParameter(String name):**Returns the parameter value for the specified parameter name.
- **public Enumeration getInitParameterNames():**Returns the names of the context's initialization parameters.
- **public void setAttribute(String name, Object object):**sets the given object in the application scope.
- **public Object getAttribute(String name):**Returns the attribute for the specified name.
- **public Enumeration getInitParameterNames():**Returns the names of the context's initialization parameters as an Enumeration of String objects.
- **public void removeAttribute(String name):**Removes the attribute with the given name from the servlet context.

Get the object of ServletContext Interface

- **getServletContext()** method of ServletConfig interface returns the object of ServletContext.
- **Syntax of getServletContext() method**
public ServletContext getServletContext()

Example:

//We can get the ServletContext object from ServletConfig object

- **ServletContext application=getServletConfig().getServletContext();**
- **ServletContext application=getServletContext();**

Syntax to provide the initialization parameter in Context scope

- The **context-param** element, subelement of web-app, is used to define the initialization parameter in the application scope.
- The param-name and param-value are the sub-elements of the context-param.
- The param-name element defines parameter name and param-value defines its value.

```
1.<web-app>
2. ....
3.
4. <context-param>
5.   <param-name>parametername</param-name>
6.   <param-value>parametervalue</param-value>
7. </context-param>
8. ....
9.</web-app>
```

InterServlet Communication 3-2

```
UserDetails.html
<HTML>
<HEAD><TITLE>User Details</TITLE></HEAD>
<BODY>
<FORM method=post action="/Example3/UserServlet1">
<H1 align=center>Welcome to Shop Stop</H1>
Please enter your user id to display your details:
<Input type=text name=txtuserid>
<br/>
<input type=submit>
</FORM>
</BODY>
</HTML>
```

Demonstration: Example 4

InterServlet Communication 3-3

```
//UserServlet2
ServletContext context=getServletContext();
Object obj=context.getAttribute("userid");

Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection con=DriverManager.getConnection("jdbc:odbc:pubs",
        "sa","playware");

Statement s=con.createStatement();
ResultSet rs=s.executeQuery("Select * from CustomerRegistration
        where UserId='"+obj.toString()+"'");
boolean flag=rs.next();

if(flag==true) {
    password=rs.getString(2);
    FullName=rs.getString(3);
    gender=rs.getString(4);
    age=rs.getString(5);
} else {
    out.println("<font color=red size=7>Login Failed</font>");
}
```

Servlet Context 2-1

- Stores the attributes and resources common to all Servlets in a `ServletContext` interface object. The methods dealing with context attributes are:

```
public Object getAttribute(String name);
```

Returns the Servlet container attribute name
Returns null if there is no attribute by that name

```
public void setAttribute(String name, Object object);
```

Binds an object to a given attribute name in the Servlet context

```
public void removeAttribute(String name);
```

Removes the attribute with the given name

Servlet Context 2-2

- ❑ The `ServletContext` interface defines methods which allow the users to access the context's state.

```
public ServletContext getContext(String path);
```

Returns an object of `ServletContext` interface that corresponds to the specified URL on the server

```
public String getInitParameter(String name);
```

Returns a string containing the context initialization parameters

```
public String getServletContextName();
```

Returns the name of the Web application corresponding to the context object as specified in the deployment descriptor file

Chat Application 5-1

```
//UserServlet
response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();

out.println("<html>");
out.println("<head><title>UserServlet</title></head>");
out.println("<body bgcolor=\"#ffffff\"onload="
    'javascript:frm.userName.focus();'>");
out.println("<FORM METHOD=POST Action='MainServlet'" +
    "name='frm'><center>Please enter your nick name<br>
    <INPUT " + "TYPE='text' NAME='userName'>" +
    "<INPUT TYPE='submit'" + "value='chat'></center></FORM>");
out.println("</body>");
out.println("</html>");
out.close();
```

Demonstration: Example 7

Chat Application 5.2

```
//MainServlet
response.setContentType("text/html;charset=UTF-8");

    out.println("<html>");
    out.println("<frameset rows='80%,*'>");
    out.println("<frame src='TopServlet?name=" +
        userName + "'>");
    out.println("<frame src='MessageServlet?name=" +
        userName + "' >");
    out.println("<frameset>");
    out.println("</body>");
    out.println("</html>");
} else {
    out.println("<h1 style='color:red'
        align='center'>" + "UserID in use</h1>");
}

}

out.close();

getServletContext().setAttribute("userArray",vec);
//Servlet Context Initialization End
```

Chat Application 5-3

```
//TopServlet
response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();
out.println("<html>");
out.println("<head><title>TopServlet</title></head>");
out.println("<frameset cols='80%, *'>");
out.println("<frame src='DisplayServlet'>");
String usrName = request.getParameter("name");
out.println("<frame src='UserList?name' = + usrName + >");
out.println("<frameset>");
out.println("</html>");
out.close();
```

Demonstration: Example 9

Chat Application 5.4

```
//UserList
response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();
out.println("<html>");
out.println("<META HTTP-EQUIV='REFRESH' CONTENT='3;Userlist'>");
out.println("<head><title>UserList</title></head>");
out.println("<body bgcolor=\"#ffffff\">");
Vector usr = (Vector) getServletContext().getAttribute("userArray");
String UserName = request.getParameter("name");
if(usr == null) {
    throw new ServletException("Error in Servlet.");
}
for(int i = 0;i<usr.size();i++) {
    if(String.valueOf(usr.get(i)).equals(UserName)) {
        out.println("<b>" + String.valueOf(usr.get(i)) +
            "</b><br>");
    } else {
        out.println(String.valueOf(usr.get(i)) + "<br>");
    }
}
out.println("</body>");
out.println("</html>");
out.close();
```

Chat Application 4.5

```
//DisplayServlet
response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();
ServletContext app = getServletContext();
StringBuffer strMsg =
    (StringBuffer) app.getAttribute("objMessage");
```

```
public void displayHtml(PrintWriter out,StringBuffer msgBuffer) {
    out.println("<html>");
    out.println("<META HTTP-EQUIV='REFRESH'
        CONTENT='3;DisplayServlet'>");
    out.println("<head><title>MessageServlet
        </title></head>");
    out.println("<body bgcolor=\"#ffffff\">");
    out.println(msgBuffer.toString());
    out.println("</body>");
    out.println("</html>");
}
```

Demonstration: Example 11

Chat Application 5-5

```
//MessageServlet
response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();

public void displayHtml(PrintWriter out) {
    out.println("<html>");
    out.println("<head><title>MessageServlet</title></head>");
    out.println("<body bgcolor=\"#ffffff\"onload =  
    'javascript:frm.message.focus();'>");
    out.println("<form method='post' name='frm'>");
    out.println("<input type='textbox'  
    name='message'size='40'>");
    out.println("<input type='submit' value='Send'>");
    out.println("</form>");
    out.println("</body>");
    out.println("</html>");
}

app.setAttribute("objMessage",objMsg);
displayHtml(out);
}
out.close();
```

Attribute

- An attribute is an object that is used to share information in a web app.
- Attribute allows Servlets to share information among themselves.
- Attributes can be SET and GET from one of the following scopes :
 - request
 - session
 - application

Attribute

Application Scope:

```
ServletContext sc=getServletContext();  
sc.setAttribute("user","Abhijit");  
sc.getAttribute("user");  
sc.removeAttribute("user");
```

context object

attribute name

attribute value

getting an attribute

removing attribute

A diagram illustrating application scope attribute operations. It shows four lines of Java code. Red arrows point from text labels to specific parts of the code: 'context object' points to 'sc' in the first line; 'attribute name' points to '"user"' in the second line; 'attribute value' points to '"Abhijit"' in the second line; 'getting an attribute' points to the second parameter '"user"' in the third line; and 'removing attribute' points to the second parameter '"user"' in the fourth line.

request Scope:

```
request.setAttribute("user","Abhijit");  
request.getAttribute("user");  
request.removeAttribute("user");
```

setting an attribute on request scope

getting an attribute

removing an attribute

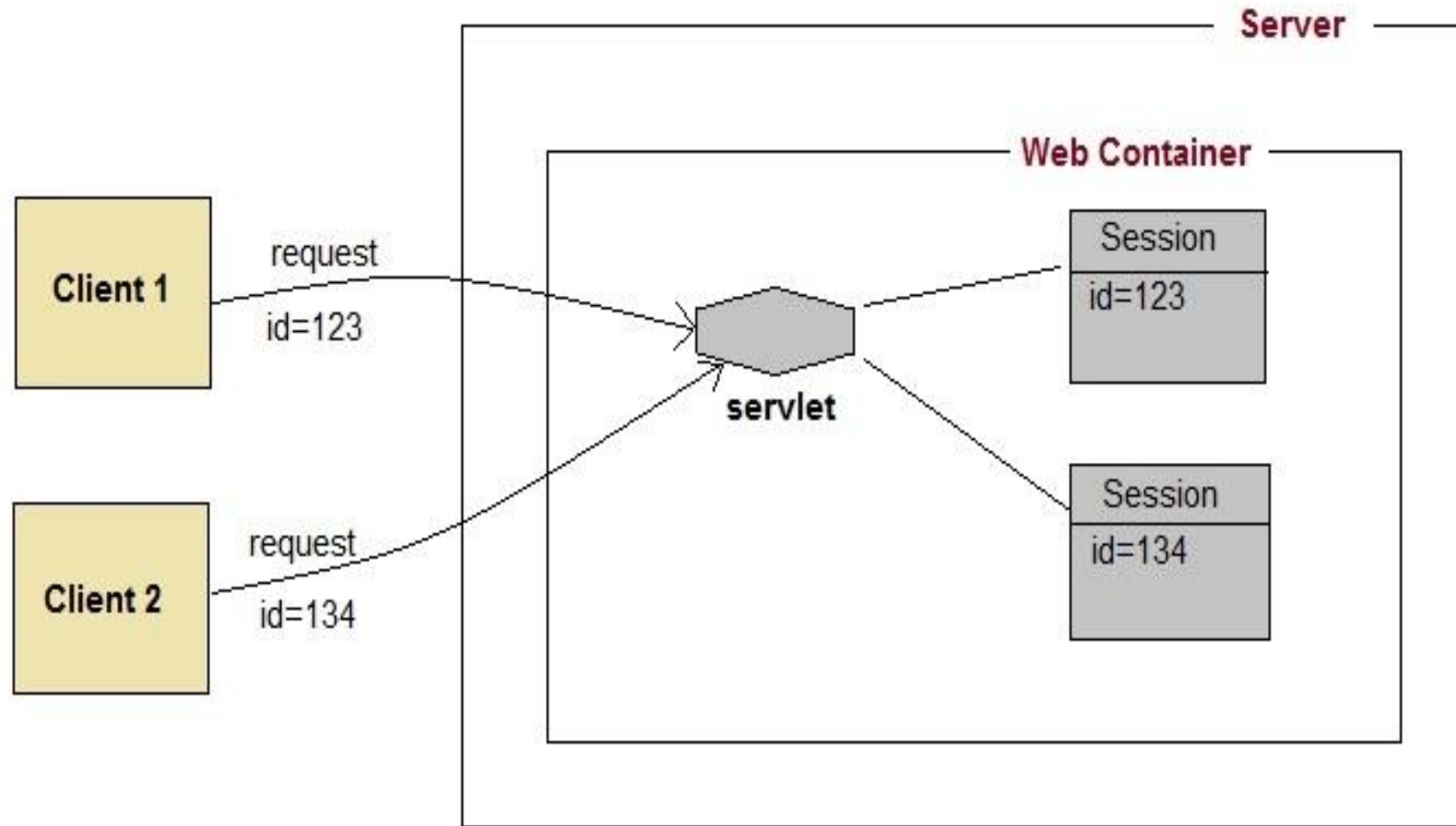
ServletRequest object

A diagram illustrating request scope attribute operations. It shows three lines of Java code. Red ovals are drawn around the word 'request' in each line. Red arrows point from text labels to the right side of each line: 'setting an attribute on request scope' points to the first line, 'getting an attribute' points to the second line, and 'removing an attribute' points to the third line. A red arrow labeled 'ServletRequest object' points to the 'request' variable in the first line, with lines extending to the 'request' variables in the other two lines.

Session

- A *session* is defined as a series of related browser requests that come from the same client during a certain time period.
- Session tracking ties together a series of browser requests—think of these requests as pages—that may have some meaning as a whole, such as a shopping cart application.

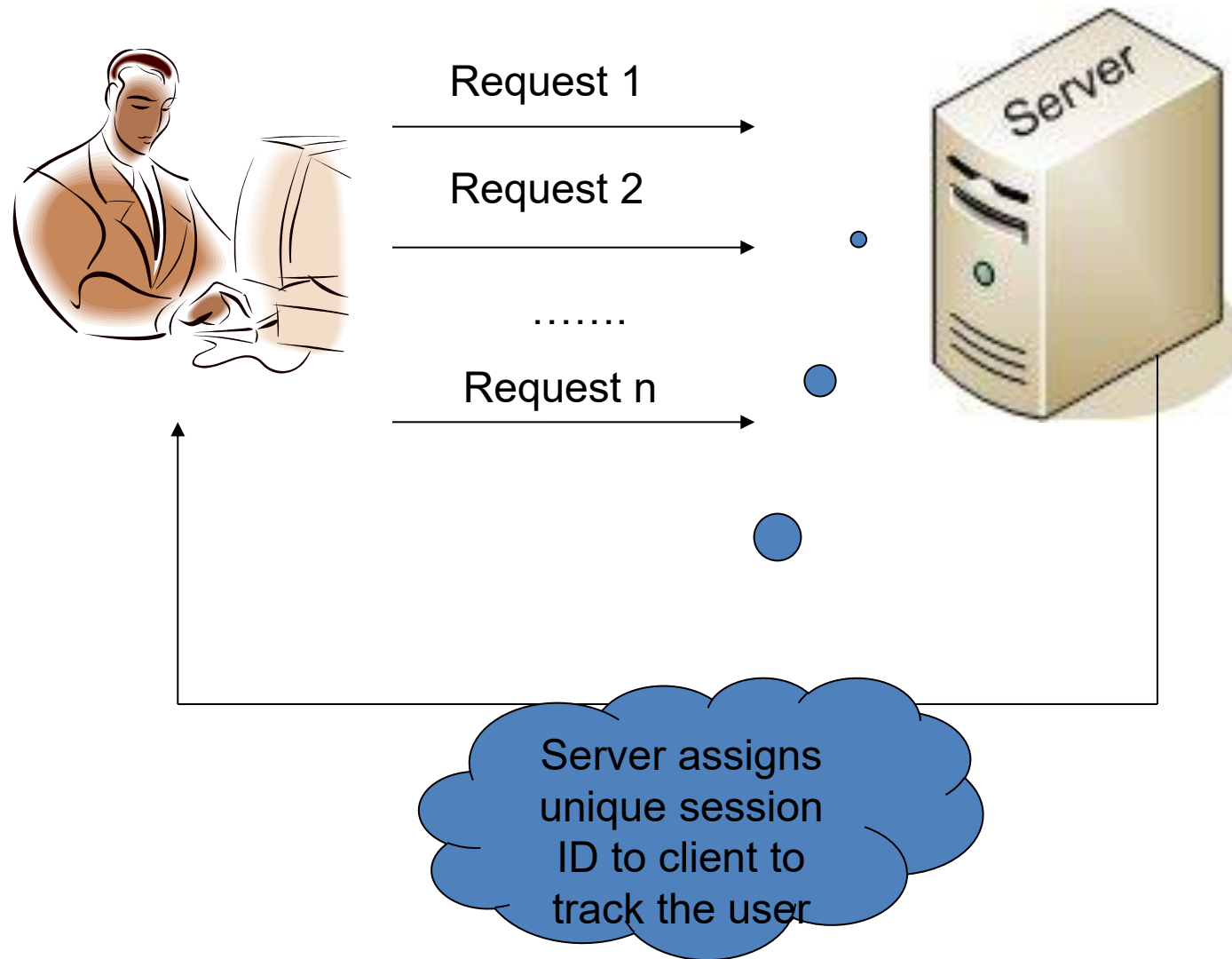
Session Tracking



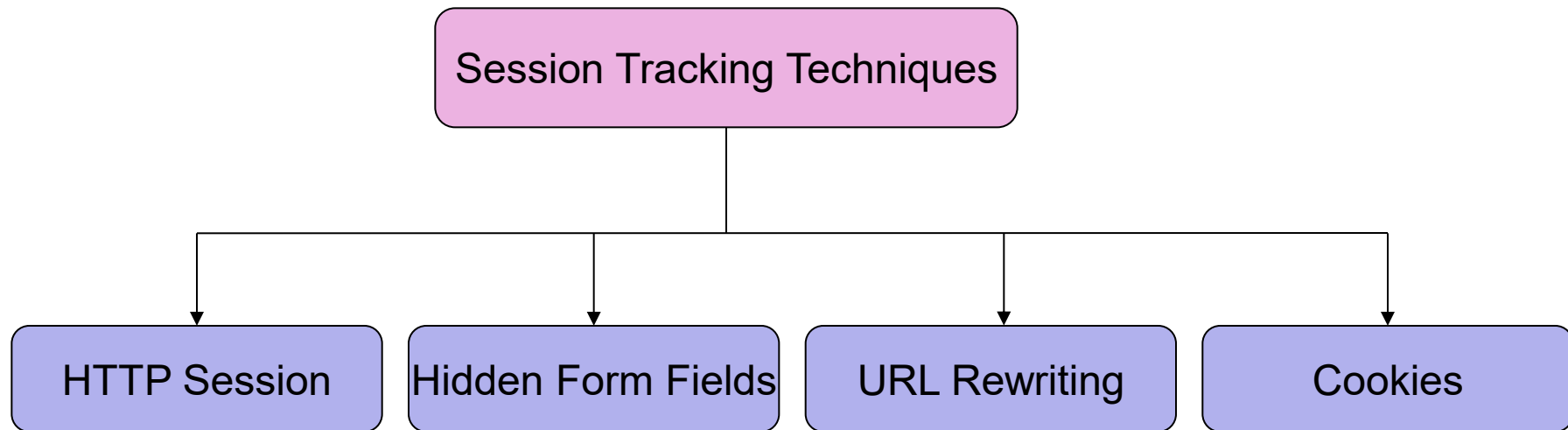
Session Tracking

- **Session** simply means a particular interval of time.
- **Session Tracking** is a way to maintain state (data) of an user. It is also known as **session management** in servlet.
- Http protocol is a stateless so we need to maintain state using session tracking techniques.
- Each time user requests to the server, server treats the request as the new request.
- So we need to maintain the state of an user to recognize to particular user.

Session Tracking



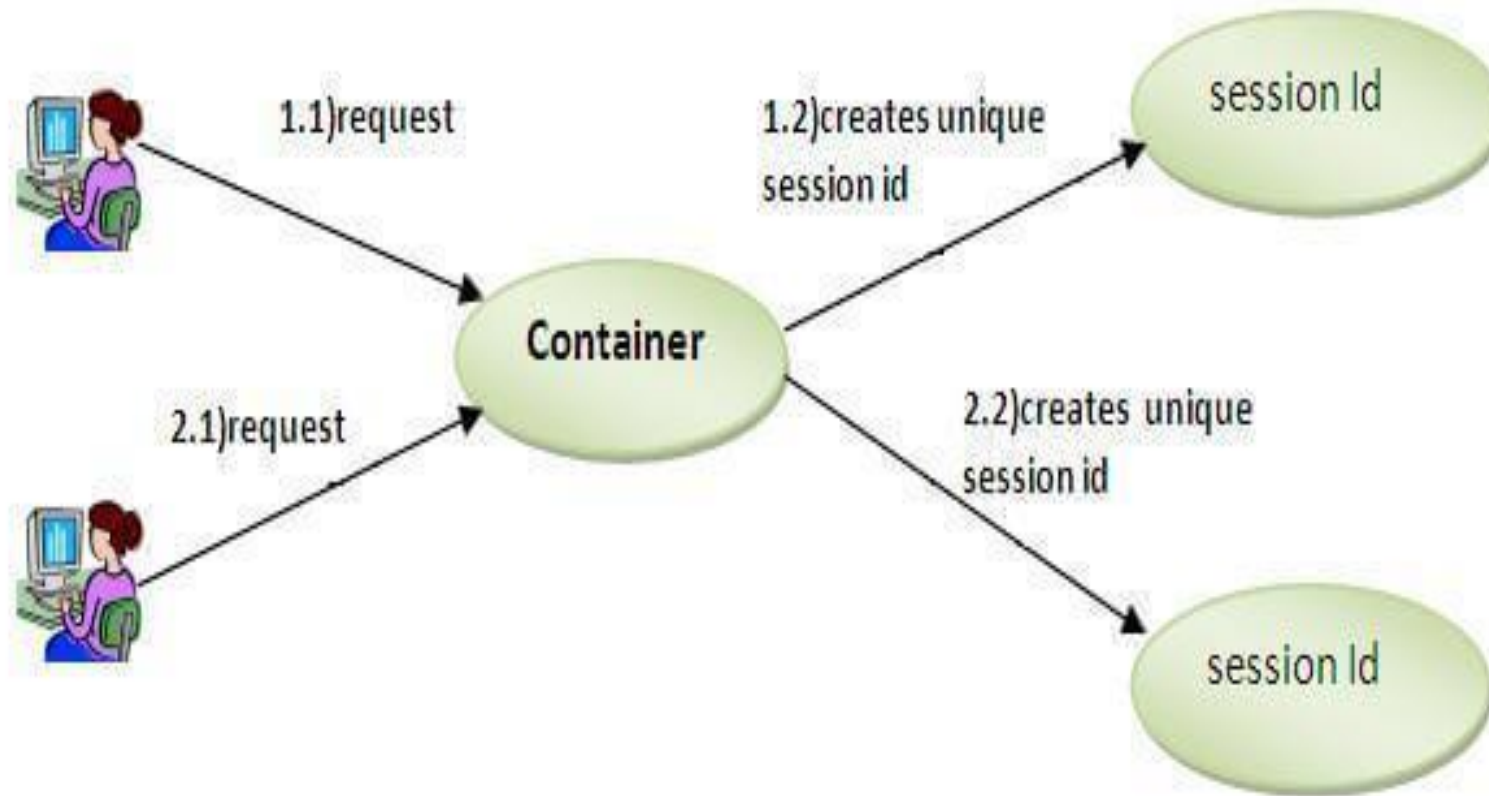
Session Tracking Techniques 9-1



HttpSession interface

- Container creates a session id for each user.
- The container uses this id to identify the particular user.
- An object of HttpSession can be used to perform two tasks:
 - Bind objects
 - View and manipulate information about a session, such as the session identifier, creation time, and last accessed time.

HTTPSession Interface - Functionality



The HttpSession Object

- Servlet provides **HttpSession** Interface which provides a way to identify a user across more than one page request or visit to a Web site and to store information about that user.
- The servlet container uses this interface to create a session between an HTTP client and an HTTP server.
- The session persists for a specified time period, across more than one connection or page request from the user.

Creating a Session

```
public HttpSession getSession();  
public HttpSession getSession(boolean value);
```

The `getSession()` method is used to create a session if there is no current session associated with the request.

- ❑ The `getSession(boolean value)` is used to create a session if the boolean value is `true` and session is currently associated with the request.
- ❑ If the boolean value is `false` then no session is associated with the current request.

Methods & Description –HTTP Session Object

public Object getAttribute(String name)

This method returns the object bound with the specified name in this session, or null if no object is bound under the name.

public Enumeration getAttributeNames()

This method returns an Enumeration of String objects containing the names of all the objects bound to this session.

public long getCreationTime()

This method returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.

public String getId()

This method returns a string containing the unique identifier assigned to this session.

public long getLastAccessedTime()

This method returns the last time the client sent a request associated with this session, as the number of milliseconds since midnight January 1, 1970 GMT.

public int getMaxInactiveInterval()

This method returns the maximum time interval, in seconds, that the servlet container will keep this session open between client accesses.

public void invalidate()

This method invalidates this session and unbinds any objects bound to it.

public boolean isNew()

This method returns true if the client does not yet know about the session or if the client chooses not to join the session.

public void removeAttribute(String name)

This method removes the object bound with the specified name from this session.

public void setAttribute(String name, Object value)

This method binds an object to this session, using the name specified.

public void setMaxInactiveInterval(int interval)

This method specifies the time, in seconds, between client requests before the servlet container will invalidate this session.

Session Tracking Techniques 9-2

User Authorization

- ☐ Tracks the user when the user logs in
- ☒ Does not allow unauthorized users to access some resources on the Web page.



Authorized user

Request



Unauthorized user

~~Request~~



Hidden Form Field

- Hidden Form Field is a **hidden (invisible) textfield** is used for maintaining the state of an user.
- We store the information in the hidden field and get it from another servlet.
- This approach is better if we have to submit form in all the pages and we don't want to depend on the browser.

Application of Hidden field

- It is widely used in comment form of a website.
- We store page id or page name in the hidden field so that each page can be uniquely identified.
- It will always work whether cookie is disabled or not.
- It is maintained at server side.
- Extra form submission is required on each pages.
- Only textual information can be used.

Session Tracking Techniques 9-3

Hidden Form Fields

- Hidden form fields are added to the HTML page. However, these fields are not rendered on the page.

```
...  
<form action = "\\firsthtml.htm" method="POST">  
<input type = "hidden" name="productid" value="123">  
</form>  
...
```

URL Rewriting

- In URL rewriting, we append a token or identifier to the URL of the next Servlet or the next resource.
- We can send parameter name/value pairs using the following format:
url?name1=value1&name2=value2&??
- A name and a value is separated using an equal = sign, a parameter name/value pair is separated from another parameter using the ampersand(&).

URL Rewriting

- When the user clicks the hyperlink, the parameter name/value pairs will be passed to the server.
- From a Servlet, we can use `getParameter()` method to obtain a parameter value.
- It will always work whether cookie is disabled or not (browser independent).
- Extra form submission is not required on each pages.
- It will work only with links.

Session Tracking Techniques 9-4

URL Rewriting

- ❑ Adds a unique session ID at the end of the URL to identify a session.
For example, following URLs are rewritten to pass session ID 10.

Original URL: `http://server:post/servlet/Rewritten`

URL rewritten with extra information:

`http://server:post/servlet/Rewritten/10`

URL rewritten with added parameters:

`http://server:post/servlet/Rewritten?sessionid=10`

URL rewritten with custom change:

`http://server:post/servlet/Rewritten;$sessionid$10`

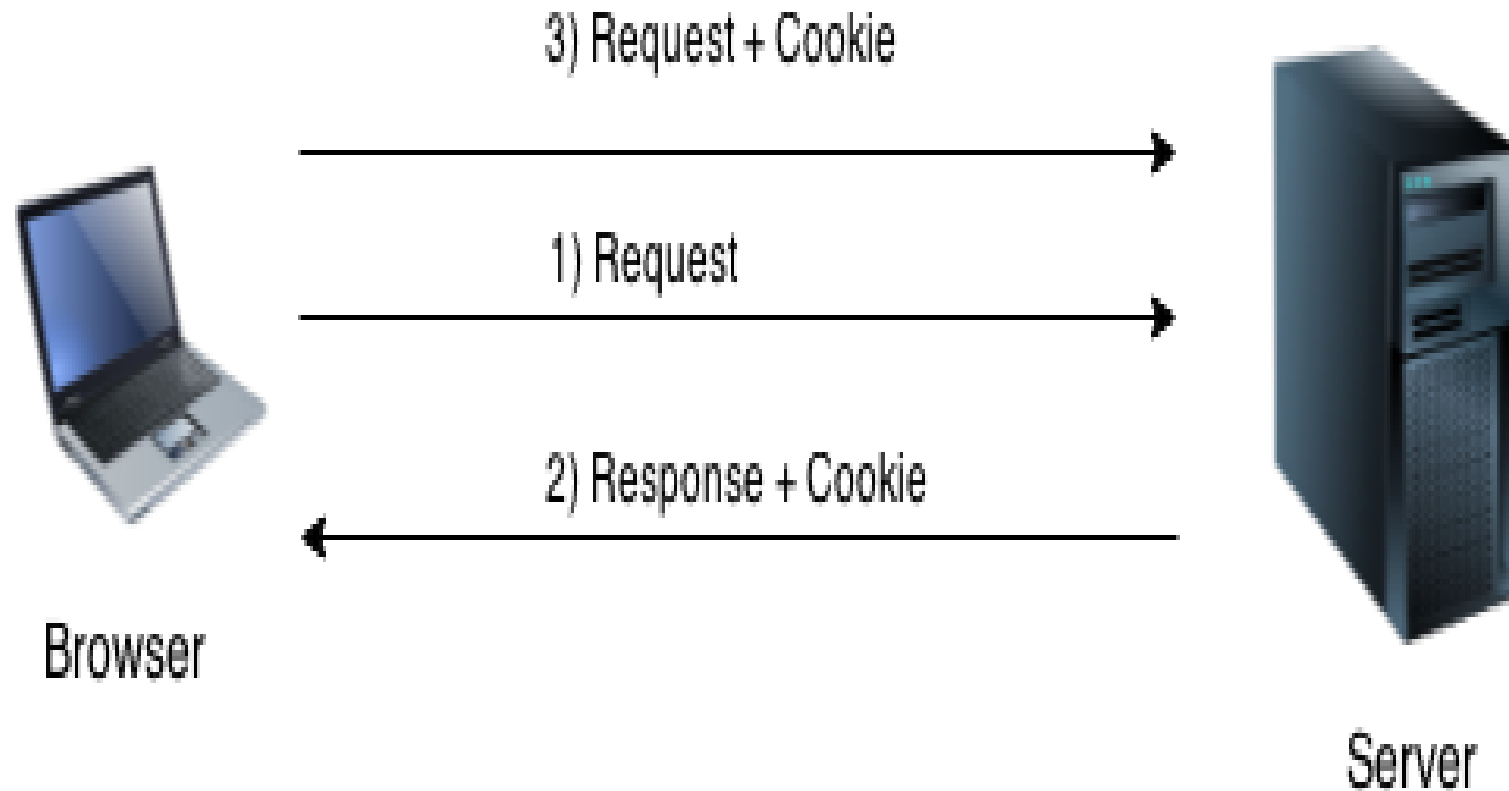
Cookies in Servlet

- A **cookie** is a small piece of information that is persisted between the multiple client requests.
- A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.

How Cookie works

- By default, each request is considered as a new request.
- In cookies technique, we add cookie with response from the servlet.
- So cookie is stored in the cache of the browser.
- After that if request is sent by the user, cookie is added with request by default.
- Thus, we recognize the user as the old user.

How Cookie works



Types of Cookie

- Non-persistent cookie

It is **valid for single session** only.

It is removed each time when user closes the browser.

- Persistent cookie

It is **valid for multiple session** .

It is not removed each time when user closes the browser.

It is removed only if user logout or signout.

Pros & Cons of Cookies

- Pros

Simplest technique of maintaining the state.

Cookies are maintained at client side.

- Cons

It will not work if cookie is disabled from the browser.

Only textual information can be set in Cookie object.

Cookie class

- **javax.servlet.http.Cookie** class provides the functionality of using cookies.
- It provides a lot of useful methods for cookies.

Constructor	Description
Cookie()	constructs a cookie.
Cookie(String name, String value)	constructs a cookie with a specified name and value.

Methods of Cookie class

Method	Description
<code>public void setMaxAge(int expiry)</code>	Sets the maximum age of the cookie in seconds.
<code>public String getName()</code>	Returns the name of the cookie. The name cannot be changed after creation.
<code>public String getValue()</code>	Returns the value of the cookie.
<code>public void setName(String name)</code>	changes the name of the cookie.
<code>public void setValue(String value)</code>	changes the value of the cookie.

Session Tracking Techniques 9-5

Cookies

- ❑ Are used to store information sent by the Web server to the client.
The server sends the cookie by setting Set-Cookie method in the response header. The syntax is as follows:

```
Set-Cookie: Name=VALUE; Comment=COMMENT; Domain=
DOMAINNAME; Max-age=SECONDS; Path=PATH; secure;
```

Name– Specifies the name of the cookie

VALUE– Specifies the value of the Cookie name

Domain – Specifies the URL for which the cookie is valid

Max-age – Specifies the life of the cookie in seconds

secure – Specifies whether cookies can be exchanged over HTTP

Session Tracking Techniques 9-6

Cookies

- ❑ Contain one or more name-value pairs, which are exchanged in request and response headers.

```
public void setDomain(String domain);
```

```
public String getDomain();
```

```
public void setMaxAge(int age);
```

```
public int getMaxAge();
```

```
public void setPath(String path);
```

```
public String getPath();
```


Session Tracking Techniques 9-7

```
//UrlRedirectServlet
response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();
    out.println("<html>");
out.println("<head>");
out.println("<title>Servlet UrlRedirectServlet</title>");
out.println("</head>");
out.println("<body>");
out.println("<h1>Servlet UrlRedirectServlet at " +
    request.getContextPath() + "</h1>");
out.println("<body bgcolor=\"#ffffff\">");
String contextPath = request.getContextPath();
String encodedUrl = response.encodeURL(contextPath +
    "/CookieReader");
out.println("<h3>This page will u
    \"necessary</h3>");
out.println("Go to the default page <a href=\"\" +
    encodedUrl + "\">here</a>.");
out.println("</body>");
out.println("</html>");

//create a cookie
Cookie c = new Cookie("servletName", getServletName());
response.addCookie(c);
out.close();
```

**getContextPath() returns context path
and encodeURL() converts the string
to URL path.**

Session Tracking Techniques 9-8

```
//CookieReader
Cookie cookie = null;
Cookie[] cookies = request.getCookies();
boolean hasCookies = false;
if (cookies != null)
    hasCookies = true;
if (hasCookies)
{
    out.println("<h2> The name and value of each found  
cookie</h2>");
    for (int i = 0; i < cookies.length; i++)
    {
        cookie = cookies[i];
        out.println("Name of cookie #" + (i + 1) +  
": " + cookie.getName() + "<br>");
        out.println("Value of cookie #" + (i + 1) +  
": " + cookie.getValue() + "<br><br>");
    }
} else {
    out.println("<h2> This request did not  
include any cookies</h2>");
}
```

If the requested URL contains cookies, the corresponding names and values are displayed.

Session Tracking Techniques 9-9

```
//SessionServlet
HttpSession session=request.getSession();
out.println("Session status:<br/>");
if(session.isNew()) {
    out.println("New session created...<br/>");
} else {
    out.println("Old session...<br/>");
}
out.println("<br/> Session id : "+session.getId());
out.println("<br/> Creation time : ");
out.println(new Date(session.getCreationTime()));
out.println("<br/> Last Accessed : ");
out.println(new Date(session.getLastAccessedTime()));
```

This example displays various session attributes as output.

Demonstration: Example 3

Tracking Service Requests

- To track service requests, include in your servlet class a field that counts the
- number of service methods that are running.
- The field should have synchronized access methods to increment, decrement, and return its value.

Tracking Service Requests

```
public class ShutdownExample extends HttpServlet {  
    private int serviceCounter = 0;  
  
    ...  
    //Access methods for serviceCounter  
    protected synchronized void enteringServiceMethod() {  
        serviceCounter++;  
    }  
    protected synchronized void leavingServiceMethod() {  
        serviceCounter--;  
    }  
    protected synchronized int numServices() {  
        return serviceCounter;  
    }  
}
```

- **The service method should increment the service counter each time the method is entered and should decrement the counter each time the method returns.**

Tracking Service Requests

```
protected void service(HttpServletRequest req,  
    HttpServletResponse resp)  
    throws ServletException, IOException {  
    enteringServiceMethod();  
    try {  
        super.service(req, resp);  
    } finally {  
        leavingServiceMethod();  
    }  
}
```

- This is one of the few times that your HttpServlet subclass should override the service method.
- The new method should call super.service to preserve all of the original service method's functionality:

Notifying Methods to Shut Down

- To ensure a clean shutdown, your destroy method should not release any shared resources until all of the service requests have completed.
- One part of doing this is to check the service counter.
- Another part is to notify the long-running methods that it is time to shut down.
- For this notification, another field is required.
- The field should have the usual access methods:

```
public class ShutdownExample extends HttpServlet
{
    private boolean shuttingDown;
    ...
    //Access methods for shuttingDown
```

```
    protected synchronized void setShuttingDown(boolean flag) {
        shuttingDown = flag;
    }
    protected synchronized boolean isShuttingDown() {
        return shuttingDown;
    }
}
```

Notifying Methods to Shut Down

- An example of the destroy method using these fields to provide a clean shutdown follows:

```
public void destroy() {  
    /* Check to see whether there are still service methods */  
    /* running, and if there are, tell them to stop. */  
    if (numServices() > 0) {  
        setShuttingDown(true);  
    }  
    /* Wait for the service methods to stop. */  
    while(numServices() > 0) {  
        try {  
            Thread.sleep(interval);  
        } catch (InterruptedException e) {  
        }  
    }  
}
```


Creating Polite Long-Running Methods

- The final step in providing a clean shutdown is to make any long-running methods behave politely.
- Methods that might run for a long time should check the value of the field that notifies them of shutdowns and should interrupt their work,
- if necessary.

```
public void doPost(...) {  
    ...  
    for(i = 0; ((i < lotsOfStuffToDo) &&  
        !isShuttingDown()); i++) {  
        try {  
            partOfLongRunningOperation(i);  
        } catch (InterruptedException e) {  
            ...  
        }  
    }  
}
```

Java XML Parser

- Java XML parser is used to work with xml data.
- XML is widely used technology to transport or store data.
- That's why there are many java xml parsers available.
- Some of the commonly used java xml parsers are;
 - DOM Parser
 - SAX Parser
 - StAX Parser
 - JAXB

Java XML Parser – DOM

- DOM Parser is the easiest java xml parser to learn.
- DOM parser loads the XML file into memory and we can traverse it node by node to parse the XML.
- DOM Parser is good for small files but when file size increases it performs slow and consumes more memory.

Read XML File

- DOM Parser to parse XML file to Object.

Write XML File

- DOM Parser to write Object data to XML file.

Edit XML File

DOM Parser can be used to edit XML data also. How to add elements, remove elements, edit element values, edit attributes in an XML document using DOM Parser.

Summary

- The successive requests made by the client can be identified using Session Tracking.
- The different session tracking techniques are: User Authorization, Hidden Form Fields, URL Rewriting and Cookies.
- The `forward()` and `include()` methods of `RequestDispatcher` objects are used by the Servlets belonging to the same application to communicate.
- The InterServlet communication techniques are: Servlet Manipulation, Servlet Reuse and Servlet Collaboration.
- A `ServletContext` is used to store the information of different Servlets.
- The `getServletContext()` method of `ServletConfig` interface is used to configure the Servlet context.